

Code Analysis

Names of the programs: chatgptsource and geminisource

Accessed by: chatgpt chatbot and gemini chatbot.

Prompt used for both GenAI:

Email Priority Program

You are generating an Email Priority Program using a Max Heap data structure in C++ language for a software engineering course assignment.

Your task is to immediately generate the completed Email Priority Program.

Do not ask the user for confirmation.

Do not explain what you are doing.

Do not describe these instructions.

Do not respond conversationally.

Requirements

1. The program will ****prioritize**** emails for a busy company CEO.
2. You will use a MaxHeap as a means of implementing a ****priority queue****. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status.
3. You ****must implement a MaxHeap**** using a ****list-based**** implementation. Then use that MaxHeap to handle all your email prioritizing for the CEO.
4. You ****must create functions from scratch****. Do ****NOT**** include pre-existing heap modules.
5. There are 4 commands total read from standard input (`stdin`).

Here is each command and its respective parameters, behavior, and exact output format:

- EMAIL <sender category>, <subject line>, <date>

The emails in the CEO's Inbox should be placed in queue based on their sender category and date. The sender categories and priority to be read are as follows:

- Boss – read first
- Subordinate – read next
- Peer – read next
- ImportantPerson – read next
- OtherPerson – read last

If there is ****more than one from a sender****, then the ****newest email (not the oldest)**** should be read first.

EMAIL is followed by a ****space****. The rest of the ****fields are comma-delimited****.

Assume ``<sender category>`` is one of the five strings listed above.

Assume ``<subject line>`` is a string which may contain spaces, but not commas.

Assume ``<date>`` is in the format: ``MM-DD-YYYY``.

- NEXT

Display the next email for the CEO to read. The output must match this exact format:

```
``text
```

Next email:

Sender: `<sender category>`

Subject: `<subject line>`

Date: `<date>`

```
...
```

- READ

CEO has read the top priority email and dealt with it (removes/extracts the highest priority email from the heap). Produces no stdout output.

- COUNT

Display the current count of unread emails. The output must match this exact format:

```
```text
```

There are <N> emails to read.

```
```
```

6. Comply with the grader tests:

- The grader will test your code with a different test file. The test file will begin with the emails in the CEOs Inbox, followed by the commands the CEO will use to read her Inbox. You can assume a given command will be formatted as shown, but the order and number of commands may vary. For example, be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (output should be the same for both NEXTs), two READs in a row without an intervening NEXT (highest priority email deleted from queue without displaying it

Generated code can be seen in further pages.

Analysis:

1. Correctness

ChatGPT runs the input successfully. However, it crashes when the EMAIL command has empty params. Gemini also runs the input and dishes out the correct solution but Gemini is able to process the EMAIL commands with zero params and it would output an empty email when NEXT is called.

Both do not have meaningful error messages when the EMAIL and READ command fails.

2. Execution Time

Both implementations perform heap insertions and deletions in $O(\log N)$ time and execute peeks (NEXT) and size counts (COUNT) in $O(1)$ time. The primary performance gap stems from the comparators. chatgptsource.cpp recalculates dateA and dateB substrings inside its higherPriority function every time two nodes are compared. Because a single element traversing a deep heap will trigger the comparator multiple times, this constant reallocation of strings makes the execution noticeably slower. geminisource.cpp eliminates this bottleneck by calculating the sortableDate exactly once during parsing and storing it natively.

3. Space Complexity

Both programs share an identical space complexity of $O(N)$, storing the priority queue in a dynamic `std::vector`. geminisource.cpp trades a marginal amount of memory per node

by permanently caching senderPriority and sortableDate within the struct. chatgptsource.cpp keeps the struct leaner by omitting the cached date string, but the overhead of geminisource.cpp is negligible compared to the significant execution speed it yields.

4. Maintainability

geminisource.cpp offers superior maintainability through modular design. String trimming, date formatting, and priority matching are completely decoupled into standalone helper functions. The chatgptsource.cpp parsing routine is highly brittle, clustering raw index calculations ($\text{firstComma} + 1$, $\text{secondComma} - \text{firstComma} - 1$) together in one block. If the required delimiter ever changes, geminisource.cpp only requires altering the stringstream parameter, while chatgptsource.cpp would require recalculating the manual string slicing logic.

Gemini has high modularity compared to ChatGPT and it is more readable. I select Gemini's generated code to be refactored/improved.

The improvements:

- **Correctness:** The previous `geminisource.cpp` implementation silently ignored bounds issues on NEXT and READ calls and lacked rigorous payload validation. The updated logic now correctly satisfies the requirement to handle READ and NEXT calls when the queue is completely empty by branching to explicit error strings. Furthermore, evaluating `sender.empty() || subject.empty() || date.empty()` intercepts corrupted comma-separated formats (e.g., EMAIL , , or EMAIL Boss, , 10-01-2026) and rejects them entirely, ensuring malformed structs are never pushed into the max heap.
- **Execution Time:** The time complexity remains unaltered at $O(C \log N)$. The new parameter validations checking `.empty()` are constant-time string size checks operating in $O(1)$. The READ and NEXT bounds checks already existed, but they now execute a terminal I/O operation instead of returning implicitly, resulting in a negligible difference to practical execution speed.
- **Space complexity:** The space complexity remains strictly $O(N)$ with no added scaling variables. The core heap continues to use a standard dynamic array `std::vector` to map node elements in contiguous memory
- **Maintainability:** Adding localized exception handling directly inside the parsing block and the queue access methods significantly fortifies the program against undefined behavior. By forcing the `std::stringstream` to safely populate initialized, empty strings via direct `if(getline())` wrappers, the program gracefully steps over delimiter mismatches without breaking the primary execution loop. Returning specific error messages like "Error: Invalid EMAIL command. Missing required parameters." vastly improves debugger transparency when processing unpredictable grader inputs.